

MAST32001 Computational Statistics - Lecture 10

Metropolis–Hastings, Mixing and Bayesian Regression

Klaus Nordhausen

University of Helsinki

WS 2026

Metropolis–Hastings

The problem

Suppose the target density is known only up to a constant:

$$\pi(\theta) = \frac{\tilde{\pi}(\theta)}{Z}, \quad Z = \int \tilde{\pi}(u) du.$$

Direct simulation from π may be difficult.

Metropolis–Hastings constructs a Markov chain using a proposal density

$$q(\theta' \mid \theta).$$

The unknown normalizing constant Z will cancel.

Metropolis–Hastings algorithm

At state θ :

1. Propose

$$\theta' \sim q(\theta' \mid \theta).$$

2. Compute

$$r = \frac{\pi(\theta')q(\theta \mid \theta')}{\pi(\theta)q(\theta' \mid \theta)}.$$

3. Accept the proposal with probability

$$\alpha(\theta, \theta') = \min(1, r).$$

4. Otherwise remain at θ .

Because $\pi = \tilde{\pi}/Z$,

$$r = \frac{\tilde{\pi}(\theta')q(\theta \mid \theta')}{\tilde{\pi}(\theta)q(\theta' \mid \theta)}.$$

Why does the acceptance probability work?

For $x \neq y$, the transition flow from x to y is

$$\pi(x)q(y | x)\alpha(x, y).$$

Using

$$\alpha(x, y) = \min \left(1, \frac{\pi(y)q(x | y)}{\pi(x)q(y | x)} \right),$$

we obtain

$$\pi(x)q(y | x)\alpha(x, y) = \min\{\pi(x)q(y | x), \pi(y)q(x | y)\}.$$

This expression is symmetric in x and y . Therefore

$$\pi(x)P(x, dy) = \pi(y)P(y, dx),$$

so detailed balance holds.

Random-walk Metropolis

A common symmetric proposal is

$$\theta' = \theta + \varepsilon, \quad \varepsilon \sim N(0, s^2).$$

Then

$$q(\theta' \mid \theta) = q(\theta \mid \theta'),$$

and the acceptance probability simplifies to

$$\alpha = \min \left(1, \frac{\pi(\theta')}{\pi(\theta)} \right).$$

The proposal scale s controls how far the chain tries to move.

Work on the log scale

Products of densities can underflow. Compute

$$\log r = \log \tilde{\pi}(\theta') - \log \tilde{\pi}(\theta) + \log q(\theta \mid \theta') - \log q(\theta' \mid \theta).$$

Generate

$$U \sim \text{Uniform}(0, 1)$$

and accept when

$$\log U < \min(0, \log r).$$

This is both numerically safer and easier to code for likelihoods.

Metropolis intuition: uphill and downhill moves

For a symmetric proposal:

- ▶ if $\pi(\theta') > \pi(\theta)$, accept with probability 1;
- ▶ if $\pi(\theta') < \pi(\theta)$, sometimes accept anyway.

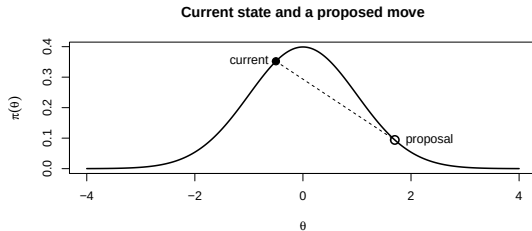
Why accept moves to lower density?

Without them, the chain would climb toward a mode and become trapped there.

The occasional downhill move is what allows the chain to represent the **whole distribution**, rather than optimize it.

MCMC is sampling, not optimization.

One-dimensional MH: visualize a proposal



The acceptance probability corrects the proposal mechanism so that the long-run distribution is the target.

Independence Metropolis–Hastings

Not every proposal must be a random walk.

An **independence proposal** has

$$q(\theta' \mid \theta) = q(\theta').$$

Then

$$\alpha(\theta, \theta') = \min \left\{ 1, \frac{\pi(\theta')q(\theta)}{\pi(\theta)q(\theta')} \right\}.$$

A good q should cover the important regions of π , including its tails.

This illustrates a general principle: proposal design should reflect the geometry of the target.

What happens after rejection?

If a proposal is rejected,

$$\theta^{(t+1)} = \theta^{(t)}.$$

Repeated states are therefore a legitimate and necessary part of an MH chain.

Deleting repeated values would change the empirical distribution of the chain and bias Monte Carlo estimates.

The chain spends more time in high-density regions partly because proposals out of those regions are more likely to be rejected.

Full R example

Example: Poisson likelihood with Gamma prior

Suppose

$$Y_i \mid \lambda \stackrel{iid}{\sim} \text{Poisson}(\lambda), \quad \lambda \sim \text{Gamma}(a, b)$$

using shape-rate parameterization.

The likelihood is

$$L(\lambda; y) \propto e^{-n\lambda} \lambda^{\sum_i y_i}.$$

The prior is

$$p(\lambda) \propto \lambda^{a-1} e^{-b\lambda}.$$

Therefore

$$p(\lambda \mid y) \propto \lambda^{a+\sum_i y_i-1} e^{-(b+n)\lambda}.$$

We know the exact posterior here, which lets us check our MCMC implementation.

Log posterior

Ignoring constants not depending on λ ,

$$\log p(\lambda \mid y) = \left(a + \sum_i y_i - 1 \right) \log \lambda - (b + n)\lambda, \quad \lambda > 0.$$

```
set.seed(101)
y_pois <- rpois(40, lambda = 3)
a_pois <- 2
b_pois <- 1
log_post_lambda <- function(lambda, y, a, b) {
  if (lambda <= 0) {
    return(-Inf)
  }
  sum(dpois(y, lambda = lambda, log = TRUE)) +
    dgamma(lambda, shape = a, rate = b, log = TRUE)
}
```

Full random-walk MH implementation

```
rw_metroplis <- function(log_target, initial, n_iter, proposal_sd, ...) {  
  draws <- numeric(n_iter)  
  draws[1] <- initial  
  accepted <- logical(n_iter)  
  
  for (t in 2:n_iter) {  
    current <- draws[t - 1]  
    proposal <- rnorm(1, mean = current, sd = proposal_sd)  
    log_ratio <-  
      log_target(proposal, ...) -  
      log_target(current, ...)  
    if (log(runif(1)) < min(0, log_ratio)) {  
      draws[t] <- proposal  
      accepted[t] <- TRUE  
    } else {  
      draws[t] <- current  
    }  
  }  
  list(draws = draws, acceptance = mean(accepted[-1]))  
}
```

Run the chain

```
set.seed(102)
fit_mh <- rw_metropolis(
  log_target = log_post_lambda,
  initial = 1,
  n_iter = 20000,
  proposal_sd = 0.7,
  y = y_pois,
  a = a_pois,
  b = b_pois
)
fit_mh$acceptance
```

```
## [1] 0.4189709
```

Run the chain

```
burn <- 2000  
lambda_draws <- fit_mh$draws[-seq_len(burn)]  
mean(lambda_draws)
```

```
## [1] 3.004494
```

```
quantile(lambda_draws, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%      97.5%
```

```
## 2.500519 2.997833 3.550854
```

Check against the exact posterior

The conjugate posterior is

$$\lambda \mid y \sim \text{Gamma} \left(a + \sum_i y_i, b + n \right).$$

```
shape_post <- a_pois + sum(y_pois)
rate_post  <- b_pois + length(y_pois)

exact_mean <- shape_post / rate_post
mcmc_mean  <- mean(lambda_draws)

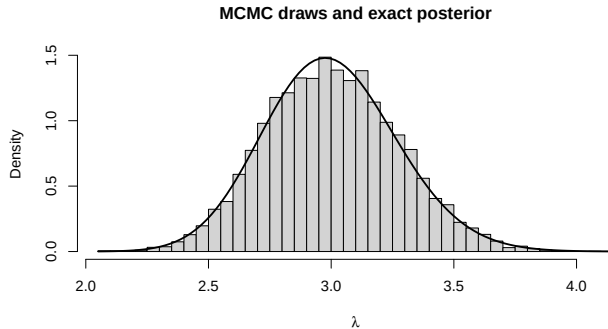
c(exact = exact_mean, MCMC = mcmc_mean)
```

```
##      exact      MCMC
## 3.000000 3.004494
```

Check against the exact posterior

```
hist(  
  lambda_draws,  
  probability = TRUE,  
  breaks = 40,  
  main = "MCMC draws and exact posterior",  
  xlab = expression(lambda)  
)  
  
curve(  
  dgamma(x, shape = shape_post, rate = rate_post),  
  add = TRUE,  
  lwd = 2  
)
```

Check against the exact posterior



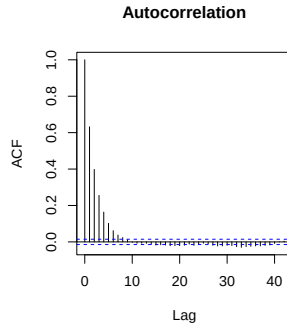
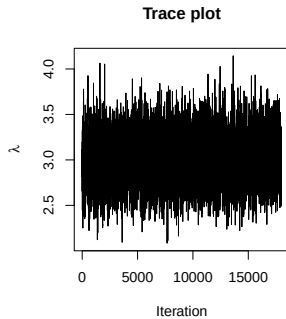
Trace plot and autocorrelation

```
plot(  
  lambda_draws,  
  type = "l",  
  xlab = "Iteration",  
  ylab = expression(lambda),  
  main = "Trace plot"  
)  
acf(  
  lambda_draws,  
  main = "Autocorrelation"  
)
```

A trace plot should look like a stable, well-explored band rather than a trend or a chain stuck for long periods.

The ACF shows the dependence between MCMC draws.

Trace plot and autocorrelation



Interactive 1: tune the proposal

Run the Poisson example with

$$s \in \{0.05, 0.7, 5\}.$$

For each proposal scale:

1. record the acceptance rate;
2. inspect the trace plot;
3. inspect the ACF;
4. estimate the posterior mean.

Before running the code, predict what will happen.

```
scales <- c(0.05, 0.7, 5)
fits <- lapply(scales,
  function(s) rw_metroplis(
    log_post_lambda, initial = 1,
    n_iter = 10000, proposal_sd = s,
    y = y_pois, a = a_pois, b = b_pois
  )
)
```

Interactive 1: what should happen?

If s is too small:

- ▶ proposals are usually accepted;
- ▶ moves are tiny;
- ▶ successive draws are highly correlated;
- ▶ exploration is slow.

If s is too large:

- ▶ proposals often land in low-density regions;
- ▶ many proposals are rejected;
- ▶ the chain repeats the same value.

A high acceptance rate is **not** automatically good.

The goal is efficient exploration, not maximum acceptance.

Interactive 1: solution – run the comparison

```
set.seed(20260812)

scales <- c(0.05, 0.7, 5)

fits_solution <- lapply(
  scales,
  function(s) rw_metroplis(
    log_post_lambda,
    initial = 1,
    n_iter = 10000,
    proposal_sd = s,
    y = y_pois, a = a_pois, b = b_pois
  )
)
```

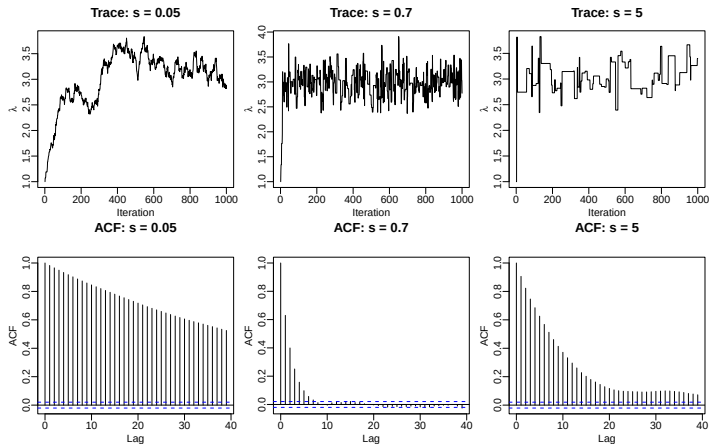
Interactive 1: solution – run the comparison

```
tuning_summary <- data.frame(  
  proposal_sd = scales,  
  acceptance = sapply(fits_solution, function(z) z$acceptance),  
  posterior_mean = sapply(  
    fits_solution,  
    function(z) mean(z$draws[-seq_len(1000)]))  
)  
round(tuning_summary, 3)
```

##	proposal_sd	acceptance	posterior_mean
## 1	0.05	0.938	3.012
## 2	0.70	0.419	3.001
## 3	5.00	0.072	2.984

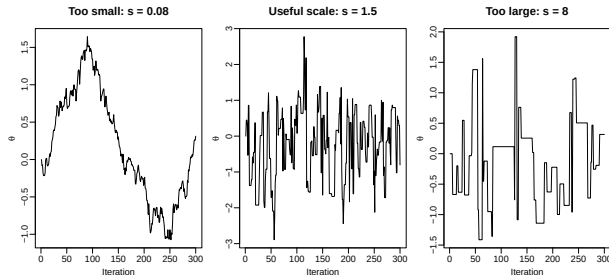
All three estimates may look plausible numerically. The important difference is **how efficiently the chains obtained them**.

Interactive 1: solution – traces and ACFs



- ▶ $s = 0.05$: very high acceptance, but tiny moves and strong autocorrelation.
- ▶ $s = 5$: many rejected proposals and repeated values.
- ▶ $s = 0.7$: a much better compromise for this example.

Proposal scale: visual comparison



All three chains target the same distribution.

The middle chain makes moves that are large enough to explore the target, but not so large that most proposals are rejected.

Acceptance rate is only a diagnostic

There is no universal optimal acceptance rate independent of algorithm, dimension and target.

For random-walk Metropolis, theoretical optimal-scaling results under specific high-dimensional assumptions motivate values around 0.234, while one-dimensional results are closer to 0.44.

These are **guides, not commandments**.

More important questions are:

- ▶ Does the chain explore all relevant regions?
- ▶ How large is the effective sample size per unit computation?
- ▶ Are results stable across chains and starting values?

Bayesian linear regression

Regression model

Consider

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2).$$

Equivalently,

$$Y_i \mid \beta_0, \beta_1, \sigma \sim N(\beta_0 + \beta_1 x_i, \sigma^2).$$

The parameter vector is

$$\theta = (\beta_0, \beta_1, \sigma).$$

Derive the regression likelihood

For one observation,

$$p(y_i \mid \beta_0, \beta_1, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left\{ -\frac{(y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma^2} \right\}.$$

Independence gives

$$L(\beta_0, \beta_1, \sigma; y, x) = \prod_{i=1}^n p(y_i \mid \beta_0, \beta_1, \sigma).$$

Therefore

$$L = (2\pi)^{-n/2} \sigma^{-n} \exp \left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \right\}.$$

Regression log-likelihood

Taking logs,

$$\ell(\beta_0, \beta_1, \sigma) = -\frac{n}{2} \log(2\pi) - n \log \sigma - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

In R this is most safely written using `dnorm(..., log=TRUE)`:

```
log_lik_reg <- function(beta0, beta1, sigma, x, y) {  
  if (sigma <= 0) {  
    return(-Inf)  
  }  
  mu <- beta0 + beta1 * x  
  sum(dnorm(  
    y,  
    mean = mu,  
    sd = sigma,  
    log = TRUE  
  ))  
}
```

Add priors

For illustration, use independent priors

$$\beta_0 \sim N(0, 10^2), \quad \beta_1 \sim N(0, 10^2), \quad \log \sigma \sim N(0, 1).$$

It is convenient to sample

$$\eta = \log \sigma$$

instead of σ , because $\eta \in \mathbb{R}$.

Define

$$\theta = (\beta_0, \beta_1, \eta), \quad \sigma = e^\eta.$$

Regression log posterior in R

```
log_post_reg <- function(theta, x, y) {  
  beta0 <- theta[1]  
  beta1 <- theta[2]  
  eta <- theta[3]  
  sigma <- exp(eta)  
  
  log_lik <-  
    sum(dnorm(y,  
      mean = beta0 + beta1 * x,  
      sd = sigma,  
      log = TRUE  
    ))  
  
  log_prior <-  
    dnorm(beta0, 0, 10, log = TRUE) +  
    dnorm(beta1, 0, 10, log = TRUE) +  
    dnorm(eta, 0, 1, log = TRUE)  
  
  log_lik + log_prior  
}
```

Multivariate random-walk MH

```
rw_metroplis_vec <- function(log_target, initial,n_iter, proposal_sd, ...) {  
  p <- length(initial)  
  draws <- matrix(NA_real_, n_iter, p)  
  draws[1, ] <- initial  
  accepted <- logical(n_iter)  
  for (t in 2:n_iter) {  
    current <- draws[t - 1, ]  
    proposal <- current + rnorm(p, sd = proposal_sd)  
    log_ratio <-  
      log_target(proposal, ...) -  
      log_target(current, ...)  
    if (log(runif(1)) < min(0, log_ratio)) {  
      draws[t, ] <- proposal  
      accepted[t] <- TRUE  
    } else {  
      draws[t, ] <- current  
    }  
  }  
  list(draws = draws, acceptance = mean(accepted[-1]))  
}
```

Worked regression example: cars

```
x_cars <- cars$speed
y_cars <- cars$dist

# Center x to improve posterior geometry
x_cars_centered <- x_cars - mean(x_cars)

set.seed(103)

fit_reg <- rw_metropolis_vec(
  log_target = log_post_reg,
  initial = c(mean(y_cars), 0, log(sd(y_cars))),
  n_iter = 30000,
  proposal_sd = c(1.2, 0.12, 0.06),
  x = x_cars_centered,
  y = y_cars
)
```

Worked regression example: cars

```
fit_reg$acceptance
```

```
## [1] 0.6933564
```

```
reg_draws <- fit_reg$draws[-seq_len(5000), ]  
colnames(reg_draws) <- c("beta0", "beta1", "log_sigma")
```

```
apply(reg_draws, 2, mean)
```

```
##      beta0      beta1 log_sigma  
## 40.966478  3.929791  2.722343
```

```
apply(reg_draws, 2, quantile, c(0.025, 0.5, 0.975))
```

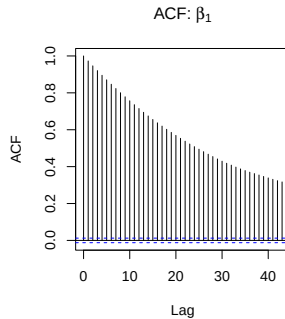
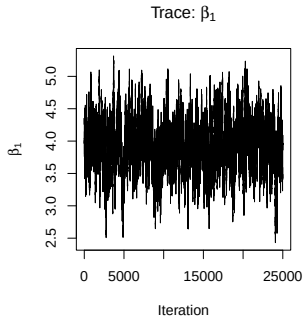
```
##           beta0      beta1 log_sigma  
## 2.5%  36.78556 3.099375  2.527207  
## 50%   40.98753 3.929300  2.720168  
## 97.5% 44.99323 4.770884  2.930166
```

Regression diagnostics

```
plot(  
  reg_draws[, "beta1"],  
  type = "l",  
  main = expression("Trace: " * beta[1]),  
  xlab = "Iteration",  
  ylab = expression(beta[1])  
)  
  
acf(  
  reg_draws[, "beta1"],  
  main = expression("ACF: " * beta[1])  
)
```

The posterior dependence among parameters also matters. Centering the predictor often reduces dependence between intercept and slope.

Regression diagnostics



Interactive 2: center or not?

Fit the same model twice:

1. with `x_cars <- cars$speed`;
2. with `x_cars_centered <- cars$speed - mean(cars$speed)`.

Use the same random-walk MH algorithm.

Compare:

- ▶ acceptance rate;
- ▶ trace plots;
- ▶ ACF of intercept and slope;
- ▶ scatterplot of posterior draws (β_0, β_1) .

Question:

Why can a statistically equivalent parameterization be computationally very different?

Interactive 2: solution – fit both parameterizations

```
set.seed(20260812)
fit_uncentered <- rw_metropolis_vec(
  log_post_reg,
  initial = c(mean(y_cars), 0, log(sd(y_cars))),
  n_iter = 30000,
  proposal_sd = c(1.2, 0.12, 0.06),
  x = x_cars,
  y = y_cars
)
set.seed(20260812)
fit_centered <- rw_metropolis_vec(
  log_post_reg,
  initial = c(mean(y_cars), 0, log(sd(y_cars))),
  n_iter = 30000,
  proposal_sd = c(1.2, 0.12, 0.06),
  x = x_cars_centered,
  y = y_cars
)
```

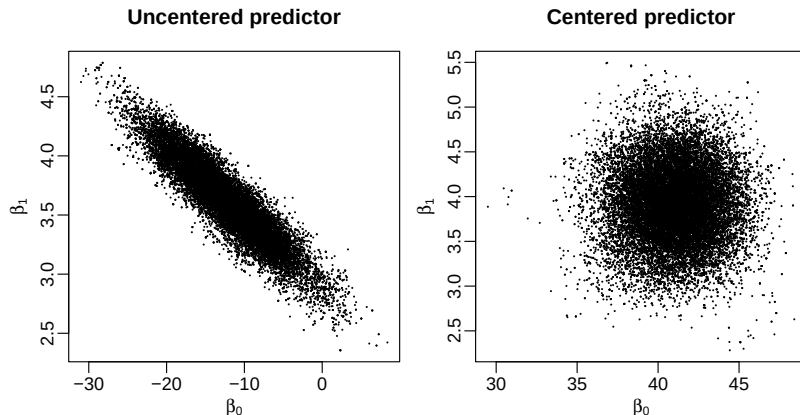
Interactive 2: solution – fit both parameterizations

```
c(  
  uncentered = fit_uncentered$acceptance,  
  centered = fit_centered$acceptance  
)
```

```
## uncentered    centered  
##  0.6023867    0.6946565
```

The statistical model has not fundamentally changed. The **geometry of the parameterization** has.

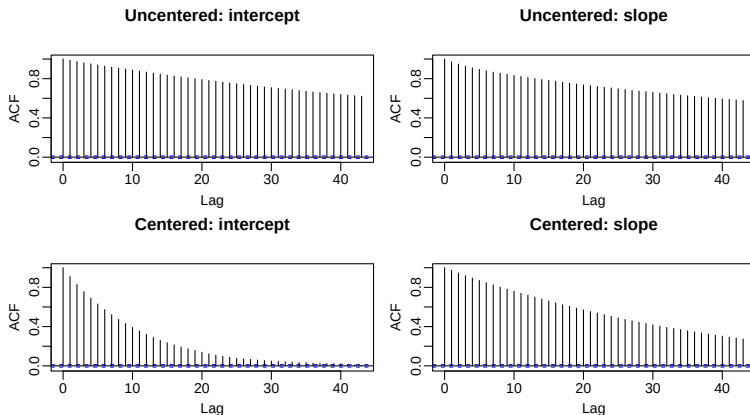
Interactive 2: solution – posterior geometry



Without centering, intercept and slope are strongly dependent.

Centering makes the posterior geometry closer to axis-aligned, so the same componentwise random-walk proposal can explore it more efficiently.

Interactive 2: solution – ACF comparison



The point is not that centering always solves every MCMC problem.

It illustrates a general principle:

parameterization can strongly affect computational efficiency.

Regression in matrix notation

With design matrix

$$X = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix},$$

the model is

$$Y \mid X, \beta, \sigma^2 \sim N_n(X\beta, \sigma^2 I_n).$$

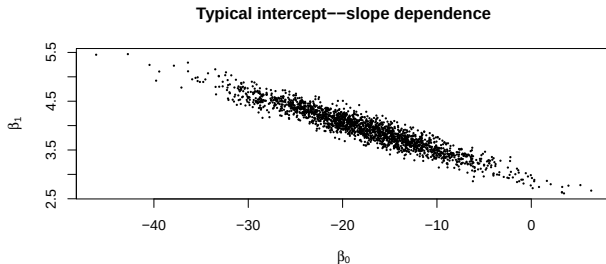
Hence

$$L(\beta, \sigma^2) \propto (\sigma^2)^{-n/2} \exp \left\{ -\frac{1}{2\sigma^2} (y - X\beta)^\top (y - X\beta) \right\}.$$

This connects the Bayesian likelihood directly to the least-squares geometry from earlier lectures.

Posterior geometry in regression

Intercept and slope can be strongly correlated when the predictor is not centered.



A random-walk proposal that ignores this geometry can waste many iterations.

Centering x rotates/reduces this posterior dependence.

Multiple chains and initialization

One long trace plot can look plausible while missing another region of the target.

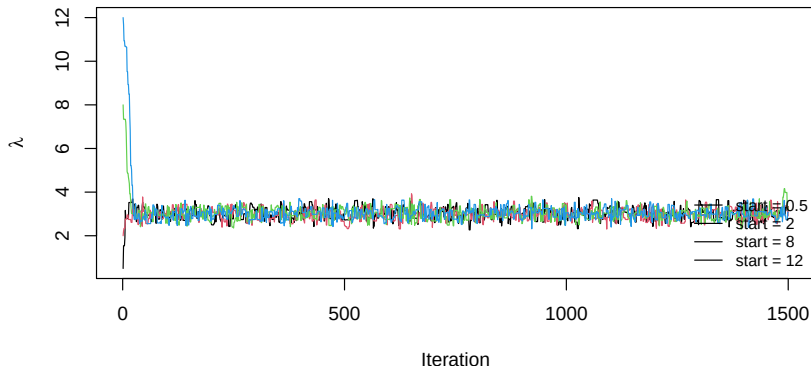
Run several chains from deliberately different starting values: $\theta_0^{(1)}, \dots, \theta_0^{(K)}$. We want them to forget their initial values and explore the same distribution.

```
set.seed(20260812)
starts <- c(0.5, 2, 8, 12)
chains <- lapply(
  starts,
  function(s) rw_metroplis(
    log_post_lambda,
    initial = s,
    n_iter = 10000,
    proposal_sd = 0.7,
    y = y_pois, a = a_pois, b = b_pois
  )$draws
)
```

Agreement between chains is evidence about convergence, not a proof.

Multiple chains: what should they look like?

Four chains from dispersed starting values



Initially the chains remember their different starting values.

After the transient phase, they should overlap and repeatedly explore the same region. This is the visual intuition behind comparing **within-chain** and **between-chain** variation.

Convergence diagnostics: what are we trying to detect?

The central difficulty is that MCMC gives us a **finite trajectory**, while our theoretical guarantees concern the long run.

Diagnostics therefore look for symptoms that the simulation has not yet become a useful representation of the target.

Typical problems are:

- ▶ chains still remember their starting values;
- ▶ chains explore different regions of the target;
- ▶ movement is extremely slow;
- ▶ successive draws are strongly dependent;
- ▶ important parts of the target have not been visited.

No diagnostic can prove convergence from a finite simulation.

The goal is instead to collect **several pieces of evidence** that the chains are behaving as expected.

Trace plots: the first diagnostic

A trace plot shows

$$\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(M)}$$

against iteration.

A useful chain should typically look like a stable band:

- ▶ no persistent upward or downward trend;
- ▶ no long-term dependence on the starting value;
- ▶ repeated movement through the same region;
- ▶ different chains should overlap.

Bad signs include:

- ▶ chains remaining separated;
- ▶ very slow drifting;
- ▶ long nearly constant stretches;
- ▶ abrupt transitions between separated regions.

The informal phrase “**hairy caterpillar**” is often used for a well-mixed trace plot.

Autocorrelation and effective sample size

MCMC draws are generally dependent.

For lag k , the autocorrelation is

$$\rho_k = \text{Corr} \left(\theta^{(t)}, \theta^{(t+k)} \right).$$

If autocorrelation decays slowly, many iterations contain similar information.

A useful summary is the effective sample size:

$$M_{\text{eff}} \approx \frac{M}{1 + 2 \sum_{k=1}^{\infty} \rho_k}.$$

Thus 10,000 stored MCMC draws need not contain information equivalent to 10,000 independent draws.

A high acceptance rate can coexist with a very small effective sample size.

Gelman–Rubin idea: compare within and between chains

Suppose we run K chains, each containing M retained draws.

If all chains are sampling the same stationary distribution, then two estimates of variability should agree:

1. **within-chain variation:** how much each chain fluctuates internally;
2. **between-chain variation:** how different the chain means are.

If chains are still stuck near different starting values, between-chain variation will be large relative to within-chain variation.

This is the basic idea behind the **Gelman–Rubin potential scale reduction factor**, usually written

$$\hat{R}.$$

Classical $\widehat{R}$

Let $\bar{\theta}_k = \frac{1}{M} \sum_{m=1}^M \theta_{km}$ be the mean of chain k , and let $\bar{\theta}_{\cdot} = \frac{1}{K} \sum_{k=1}^K \bar{\theta}_k$. The average within-chain variance is

$$W = \frac{1}{K} \sum_{k=1}^K s_k^2.$$

The between-chain variance is

$$B = \frac{M}{K-1} \sum_{k=1}^K (\bar{\theta}_k - \bar{\theta}_{\cdot})^2.$$

Estimate the marginal variance by

$$\widehat{V} = \frac{M-1}{M} W + \frac{1}{M} B.$$

Then a basic version is

$$\widehat{R} = \sqrt{\frac{\widehat{V}}{W}}.$$

How should we interpret $\hat{R}$?

If the chains have mixed well, their means should be similar and

$$B$$

should no longer be large relative to

$$W.$$

Then

$$\hat{R} \approx 1.$$

If

$$\hat{R} > 1,$$

running the chains longer may still substantially change our estimate of the target distribution.

Historically, values such as 1.1 were sometimes used as rough cutoffs. Modern practice is more demanding; values below about 1.01 are often sought.

Modern implementations also use **split and rank-normalized** $\hat{R}$, which are more robust than the classical formula shown here.

Do not use a threshold mechanically: $\hat{R}$ is one diagnostic among several.

R: a simple classical $\hat{R}$

```
rhat_basic <- function(chains) {  
  # columns = chains, rows = retained iterations  
  M <- nrow(chains)  
  K <- ncol(chains)  
  
  chain_means <- colMeans(chains)  
  W <- mean(apply(chains, 2, var))  
  B <- M * var(chain_means)  
  
  V_hat <- ((M - 1) / M) * W + B / M  
  sqrt(V_hat / W)  
}
```

For serious analyses, use established software rather than implementing $\hat{R}$ yourself.

The function is merely an illustration.

Example: diagnostics detect a bad MCMC run

Use the Poisson–Gamma target from earlier, but deliberately choose a proposal that is too small.

```
set.seed(20260812)
starts <- c(0.5, 2, 8, 12)
bad_chains <- sapply(starts,
  function(s) rw_metropolis(
    log_post_lambda,
    initial = s,
    n_iter = 1500,
    proposal_sd = 0.03,
    y = y_pois, a = a_pois, b = b_pois
  )$draws
)
rhat_basic(bad_chains)
```

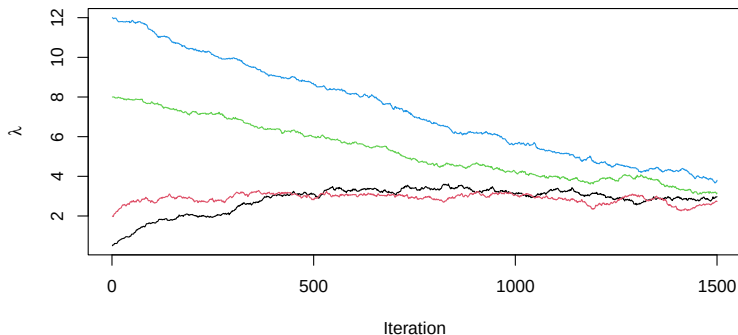
```
## [1] 1.795316
```

The proposal is accepted very frequently, but each accepted move is tiny.

So **high acceptance does not imply good mixing**.

What do the bad chains look like?

Proposal SD = 0.03: chains remember their starts



The problem is visible before calculating any statistic:

- ▶ chains started far apart remain separated;
- ▶ exploration is slow;
- ▶ the starting values are still visible;
- ▶ $\hat{R}$ is correspondingly inflated.

Fix the problem: improve the proposal and run again

Increase the proposal scale so that the chain can make meaningful moves.

```
set.seed(20260812)
good_chains <- sapply(
  starts,
  function(s) rw_metropolis(
    log_post_lambda,
    initial = s,
    n_iter = 10000,
    proposal_sd = 0.7,
    y = y_pois, a = a_pois, b = b_pois
  )$draws[-seq_len(2000)]
)
rhat_basic(good_chains)
```

```
## [1] 1.000045
```

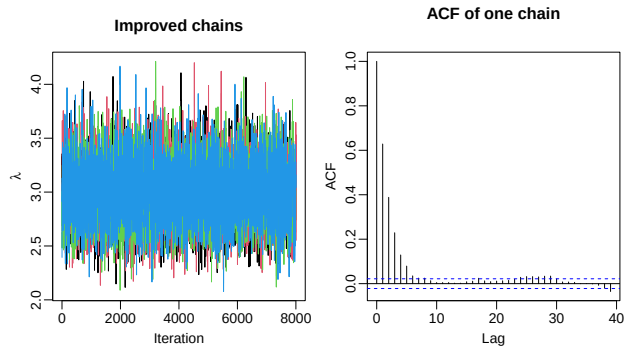
Fix the problem: improve the proposal and run again

We changed two things:

1. a proposal scale that explores the posterior more efficiently;
2. a longer run, discarding the initial transient period.

The important fix is not simply “run longer”: first improve the algorithm if mixing is poor.

After the fix: compare diagnostics



Now we want several diagnostics to tell a consistent story:

- ▶ trace plots overlap;
- ▶ chains no longer visibly remember their starts;
- ▶ autocorrelation decays reasonably;
- ▶ $\hat{R}$ is close to 1;
- ▶ effective sample size is adequate for the desired precision.

Monte Carlo error of an MCMC estimate

Suppose we estimate

$$E_{\pi}\{h(\theta)\}$$

using the chain average.

The relevant uncertainty is the **Monte Carlo standard error**

$$\text{MCSE}(\bar{h}) \approx \sqrt{\frac{\widehat{\text{Var}}_{\pi}(h)}{M_{\text{eff}}}}.$$

This separates two questions:

1. posterior uncertainty about the parameter;
2. numerical uncertainty because we used a finite MCMC run.

Running the chain longer reduces the second, not the first.

Diagnostics: put the evidence together

At minimum, inspect:

- ▶ **trace plots:** do chains overlap and explore a stable region?
- ▶ **multiple dispersed chains:** have they forgotten their initial values?
- ▶ $\hat{R}$: is between-chain variation consistent with within-chain variation?
- ▶ **autocorrelation:** how persistent is dependence?
- ▶ **effective sample size:** how much independent-equivalent information was obtained?
- ▶ **MCSE:** is the simulation accurate enough for the quantity being reported?
- ▶ **stability:** do posterior summaries remain similar when the run is extended?

There is no single universal “convergence test.”

A convincing analysis uses several complementary diagnostics.

Later, Stan will add diagnostics specific to HMC/NUTS, such as divergent transitions.

Burn-in, warmup and thinning

Early iterations may reflect initialization rather than the target distribution.

In basic MCMC we often discard an initial **burn-in** period.

Modern adaptive algorithms use **warmup** both to approach the typical set and to tune algorithmic parameters.

Thinning means retaining only every k th draw. It is usually not a remedy for poor mixing: it throws away samples rather than improving the chain.

Summary

Metropolis–Hastings:

- ▶ needs the target only up to proportionality;
- ▶ corrects an easy proposal using an acceptance probability;
- ▶ satisfies detailed balance;
- ▶ can be implemented with only a log target;
- ▶ may mix very poorly if proposal scale or parameterization is poor.

For regression, the likelihood follows directly from

$$Y_i \mid x_i, \beta, \sigma \sim N(x_i^\top \beta, \sigma^2).$$